

Perfetto 👍 ti lascio una **policy semplice e pratica** per una libreria ESP32 come la tua (lnLogger).

Così non devi pensarci ogni volta.



Regole SemVer pratiche per una libreria C++ (ESP32)

Formato:

MAJOR.MINOR.PATCH



MAJOR → rompi la compatibilità

Incrementa **MAJOR** quando il codice degli utenti deve cambiare.

Esempi concreti:

- Cambi nome a una macro pubblica
- Modifichi la firma di una funzione pubblica
- Rimuovi una funzione
- Cambi comportamento in modo incompatibile
- Cambi namespace pubblico

Esempio:

```
// PRIMA  
LOG_DEBUG("msg");
```

```
// DOPO  
LN_LOG_DEBUG("msg");
```

→ 1.4.2 → 2.0.0



MINOR → aggiungi funzionalità compatibili

Incrementa **MINOR** quando:

- Aggiungi nuove macro
- Aggiungi nuove funzioni

- Aggiungi parametri opzionali
- Migliori performance
- Aggiungi supporto a nuova feature ESP32

Esempio:

```
LOG_INFO("msg");    // già esistente  
LOG_WARN("msg");    // nuova macro
```

→ 1.4.2 → 1.5.0

PATCH → bugfix o miglioramenti interni

Incrementa **PATCH** quando:

- Fix di bug
- Refactoring interno
- Miglioramenti interni
- Commenti/documentazione
- Ottimizzazioni senza cambiare API

→ 1.4.2 → 1.4.3

Regola mentale veloce

Fatti sempre questa domanda:

Un utente deve modificare il suo codice dopo l'upgrade?

-  Sì → MAJOR
-  No → continua

Hai aggiunto roba nuova?

-  Sì → MINOR
 -  No → PATCH
-

Esempi realistici per InLogger

Caso 1

Ottimizzi macro interne → PATCH

Caso 2

Aggiungi supporto colore su seriale → MINOR

Caso 3

Cambi il modo in cui si inizializza il logger → MAJOR

Consiglio professionale

Per librerie embedded:

- La **stabilità API** è sacra
- Evita MAJOR troppo frequenti
- Usa PATCH spesso
- Usa MINOR quando aggiungi feature vere

Molte librerie mature stanno anni in 1.x.x.

Se vuoi, possiamo anche rendere il tuo script più intelligente:

- Se modifichi solo .cpp → suggerisce PATCH
- Se modifichi .h pubblici → suggerisce MINOR
- Se rimuovi simboli → avviso MAJOR

Ma quello diventa livello "release engineering serio" 😎